Module 07 — Database Programming Bridge

Phase: Advanced Python
Previous: MySQL for Full Stack Developers
Next: HTML (shared FRONTEND)

Module Outcome

Connect Python to MySQL, run SQL through a cursor, and keep user input out of the SQL string with parameterized queries.

By the end of this module you will be able to:

- Explain why a shared app needs a database, not only CSV/JSON files
- Connect, get a cursor, `execute`, `commit` writes, and close
- Use `%s` placeholders (never f-string SQL from user input)
- Map INSERT / SELECT / UPDATE / DELETE to `execute` + `fetch` / `commit`

Requires MySQL: Examples that call `mysql.connector.connect` need a running server and a practice database. Use kit 02_mysql_library_seed (creates `training_db`) or a `students` table you create yourself. They will **not** run as standalone Python on a machine without MySQL.

1. Why This Concept Matters

You already save data in **files** (CSV, JSON). That works for one user on one machine. A web application has **many users at once** — clerks updating stock, students placing orders. Shared data must live in a **database server** (MySQL).

Python's job: validate input, talk to MySQL, return rows to the rest of the app (CLI now; UI later).

Files	Database
Fine for one laptop / homework	Many users, concurrent writes
You invent search/filter	SQL queries, indexes, relations
Easy to corrupt with partial writes	Transactions (`commit` / `rollback`)

SQL injection: an attacker types malicious SQL into a form. Never build SQL with f-strings from user input. Always use **parameterized queries** (template + separate values).

2. Architecture — who talks to whom

Understand

```
[Python script / app]
        ↓  mysql-connector  or  PyMySQL
[MySQL server]
```

```
        ↓
[training_db  →  tables: students, books, ...]
```

Lifecycle:

1. **Connect** — open a session (host, user, password, database).
2. **Cursor** — run SQL and fetch rows.
3. **Execute** — send statements (`SELECT …`, `INSERT …`).
4. **Commit** — make INSERT / UPDATE / DELETE permanent.
5. **Close** — free server resources.

Skip `commit` and writes vanish. Leave connections open and the server runs out of slots.

How It Works

The connector is a library. Your script does not store the tables itself — it **sends SQL** over a connection to MySQL, then reads rows back.

```
fee_logic.py  →  TCP :3306  →  training_db  →  roll, name, marks
```

⚠️ WATCH OUT

Closing only the cursor is not enough. Close the **connection** too (a `finally` block, or a context manager if you use one).

Try It

On paper, list the five lifecycle steps in order. Circle which step makes an INSERT visible to the next program that connects.

---

3. Setup — pick one connector

Understand

This course uses **`mysql-connector-python`** (Oracle's connector) as the lab standard. **PyMySQL** is a pure-Python alternate. Both use `%s` placeholders. Pick **one** for a project — do not mix.

```
python -m venv venv
source venv/bin/activate
pip install mysql-connector-python
# OR: pip install PyMySQL
pip freeze > requirements.txt
```

How It Works

After the package is installed in your venv:

```
import mysql.connector

print("Import OK — connector installed in this venv")
print("Version:", mysql.connector.__version__)
```

Expected Output (version number depends on what you installed):

```
Import OK — connector installed in this venv
Version: <installed version>
```

This import check does **not** need a running MySQL server. Connecting does.

⚠ WATCH OUT

```
# Wrong — sqlite-style placeholder
cursor.execute("SELECT * FROM students WHERE roll = ?", (roll,))

# Prefer — MySQL connectors use %s
cursor.execute("SELECT * FROM students WHERE roll = %s", (roll,))
```

Try It

In your venv, `pip install mysql-connector-python`, then run `import mysql.connector` and print `__version__`. Do not mix PyMySQL in the same project.

---

4. Connection parameters

Understand

```
config = {
    "host": "localhost",
    "user": "root",           # use a limited user in production
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}
```

Production: store credentials in **environment variables**, not source code.

How It Works

Requires live MySQL and a database named `training_db` (create it in Workbench, or run kit `02_mysql_library_seed`).

```
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",                 # replace with lab user
    "password": "your_password",    # replace with lab password
    "database": "training_db",
    "port": 3306,
}

conn = mysql.connector.connect(**config)
cursor = conn.cursor()
cursor.execute("SELECT DATABASE();")
print("Current DB:", cursor.fetchone()[0])
cursor.close()
```

```
conn.close()
print("Closed cleanly")
```

Sample output when MySQL is up (not runnable without the server):

```
Current DB: training_db
Closed cleanly
```

Safer: read the password from the environment.

```
import os
import mysql.connector

password = os.environ.get("MYSQL_PASSWORD")
if not password:
    raise SystemExit("Set MYSQL_PASSWORD first, e.g.\n  export MYSQL_PASSWORD='your_lab_password'")

config = {
    "host": "localhost",
    "user": "root",
    "password": password,
    "database": "training_db",
    "port": 3306,
}
conn = mysql.connector.connect(**config)
print("Connected without a hardcoded password")
conn.close()
```

⚠️ WATCH OUT

```
# Wrong — password committed to Git
password = "MySecret123"

# Prefer
password = os.environ.get("MYSQL_PASSWORD")
```

Catch `mysql.connector.Error` on connect so a down server prints a clear message, not a mystery traceback. Always close in `finally` if you do not use a context manager.

Try It

Requires live MySQL. Connect with your lab user, `SELECT DATABASE();`, print the name, close cursor and connection.

---

5. Parameterized queries vs SQL injection

Understand

The driver sends the SQL **template** and the **values** separately. User input cannot change the query structure. That is the #1 SQL-injection defence in Python + MySQL.

```
# SAFE
sql = "SELECT * FROM students WHERE roll = %s"
cursor.execute(sql, (roll,))
```

```
# UNSAFE — NEVER
sql = f"SELECT * FROM students WHERE roll = {roll}"
```

A trailing comma in `(roll,)` makes a one-element **tuple**. `execute` needs a sequence of params.

How It Works

This comparison is **strings only** — no MySQL required:

```
user_input = "1 OR 1=1"

safe_sql = "SELECT * FROM students WHERE roll = %s"
# cursor.execute(safe_sql, (user_input,))  # looks for roll literally '1 OR 1=1'

unsafe_sql = f"SELECT * FROM students WHERE roll = {user_input}"
# cursor.execute(unsafe_sql)  # WHERE roll = 1 OR 1=1  →  every row

print("Safe pattern uses %s + tuple:", safe_sql)
print("Unsafe f-string expands to:", unsafe_sql)
print("Lesson: f-string injection broke row filtering — always use %s.")
```

Expected Output:

```
Safe pattern uses %s + tuple: SELECT * FROM students WHERE roll = %s
Unsafe f-string expands to: SELECT * FROM students WHERE roll = 1 OR 1=1
Lesson: f-string injection broke row filtering — always use %s.
```

Requires live MySQL plus kit 20_sql_injection_demo to see the same idea on a login query. Finish `build_query_safe` in `starter/login_query.py`. Do not copy `solution/`.

⚠ WATCH OUT

```
# Wrong
sql = f"SELECT * FROM users WHERE username='{username}'"

# Prefer
sql = "SELECT * FROM users WHERE username = %s"
cursor.execute(sql, (username,))
```

Attacker input `admin' --` can comment out the password check if you concatenate.

Try It

Print an unsafe f-string using `username = "admin' --"` and a `users` login query. Then write the `%s` version next to it. Which one still contains the quote-and-comment as **data** rather than SQL?

---

6. CRUD and fetch

Understand

Operation	SQL	Python
Create	INSERT	`execute` + `commit`

Operation	SQL	Python
Read	SELECT	`execute` + `fetchone` / `fetchall`
Update	UPDATE	`execute` + `commit`
Delete	DELETE	`execute` + `commit`

`fetchone()` → one row (or `None`). `fetchall()` → a list of rows. `cursor.rowcount` after a write → how many rows were affected.

Name columns in SELECT when you can (`roll, name, marks`) instead of `SELECT *`.

How It Works

Create the practice table **once** in Workbench or `mysql` (this is SQL, not Python):

```
CREATE DATABASE IF NOT EXISTS training_db;
USE training_db;

CREATE TABLE IF NOT EXISTS students (
    roll INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    branch VARCHAR(50),
    marks DECIMAL(5,2)
);
```

Requires live MySQL and that `students` table:

```
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}

roll = 301
conn = mysql.connector.connect(**config)
cur = conn.cursor()

cur.execute(
    "INSERT INTO students (roll, name, branch, marks) VALUES (%s, %s, %s, %s)",
    (roll, "Vikram", "ECE", 70.0),
)
conn.commit()

cur.execute("SELECT roll, name, marks FROM students WHERE roll = %s", (roll,))
print("After INSERT:", cur.fetchone())

cur.execute("UPDATE students SET marks = %s WHERE roll = %s", (85.0, roll))
conn.commit()

cur.execute("SELECT roll, name, marks FROM students WHERE roll = %s", (roll,))
print("After UPDATE:", cur.fetchone())

cur.execute("DELETE FROM students WHERE roll = %s", (roll,))
conn.commit()
```

```
print("Deleted rows:", cur.rowcount)

cur.close()
conn.close()
```

Sample output when MySQL is up:

```
After INSERT: (301, 'Vikram', Decimal('70.00'))
After UPDATE: (301, 'Vikram', Decimal('85.00'))
Deleted rows: 1
```

(`Decimal` vs `70.0` depends on the column type. The **shape** is roll, name, marks.)

On failure after a write, `conn.rollback()` undoes the transaction, then close.

Library seed (`02_mysql_library_seed`) already has a `books` table. Filter with `%s`:

```
# Requires live MySQL + 02_mysql_library_seed (database training_db)
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}

author = "Luciano Ramalho"  # matches the seed

conn = mysql.connector.connect(**config)
cur = conn.cursor()
cur.execute(
    "SELECT title, author FROM books WHERE author = %s",
    (author,),
)
for row in cur.fetchall():
    print(row)

cur.close()
conn.close()
```

Sample output when the seed is loaded:

```
('Fluent Python', 'Luciano Ramalho')
```

Optional: `conn.cursor(dictionary=True)` so each row is a dict (`row["title"]`) instead of a tuple.

⚠ WATCH OUT

```
# Wrong — change never saved
cur.execute("INSERT INTO students ...", (...))
# forgot conn.commit()

# Prefer
cur.execute(...)
conn.commit()
```

On error in a write path: `conn.rollback()`, then tell the user the change was not saved.

Try It

Requires live MySQL. Insert one student with `%s` and a tuple, `commit()`, print `cursor.rowcount`. Then SELECT that roll with a parameterized `WHERE`.

Step-by-Step: Build It

Requires live MySQL and the `students` table from section 6. Type connect → parameterized INSERT → `commit` → SELECT. Catch a connector error and always close.

```
import mysql.connector
from mysql.connector import Error

config = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
}

conn = None
cursor = None
try:
    conn = mysql.connector.connect(**config)
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO students (roll, name, branch, marks) VALUES (%s, %s, %s, %s)",
        (101, "Ananya", "CSE", 88.5),
    )
    conn.commit()
    print("Rows inserted:", cursor.rowcount)
    cursor.execute(
        "SELECT roll, name, marks FROM students WHERE roll = %s",
        (101,),
    )
    print("Row:", cursor.fetchone())
except Error as err:
    if conn:
        conn.rollback()
    print("DB error:", err)
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()
        print("Closed")
```

Sample output when MySQL is up (a second run will fail on duplicate primary key — catch it and print the error):

```
Rows inserted: 1
Row: (101, 'Ananya', Decimal('88.50'))
Closed
```

Then open the kits: 02_mysql_library_seed (run the SQL seed; fill `starter/queries.sql`) and 20_sql_injection_demo (parameter discipline). Do not copy `solution/`.

Practice Exercises

Attempt each problem before reading the model answer. Items 2–5 **require live MySQL** unless noted.

1. (No MySQL.) Print a **bad** login query that concatenates `username = "admin' --"` into SQL, then print the safe `%s` template beside it.

Model answer:

```
username = "admin' --"
print("unsafe:", f"SELECT * FROM users WHERE username='{username}' AND password='x'")
print("safe:   SELECT * FROM users WHERE username = %s AND password = %s")
```

Expected Output:

```
unsafe: SELECT * FROM users WHERE username='admin' --' AND password='x'
safe:   SELECT * FROM users WHERE username = %s AND password = %s
```

2. **Requires live MySQL.** Connect and print the server version (`SELECT VERSION()`). Insert 3 students with parameterized INSERT (`executemany` is fine). `commit()`.

Hint: `cursor.executemany(sql, list_of_tuples)`.

3. **Requires live MySQL.** `SELECT name, marks FROM students WHERE marks > %s` with bound value `75`. Print each row.
4. Write `get_student(conn, roll)` that returns a dict or `None`, and `update_marks(conn, roll, new_marks)` that `commit`s and returns `rowcount`. Use `%s` only.

Hint: `cursor(dictionary=True)` then `fetchone()`.

5. Sketch a menu CLI: 1 Add · 2 List · 3 Update · 4 Delete · 5 Exit. Every query parameterized. On a failed UPDATE, `rollback` and print that the change was not saved. (You may implement this as the mini-project.)

Lab

Connect, parameterize, and fetch. **Requires live MySQL** and kit `02_mysql_library_seed` (or the `students` table). This is not the full CLI mini-project.

Problem

A librarian wants books by one author without building SQL by hand. A missing server must print a clear message.

What to Build

A short script: connect to `training_db`, `SELECT title, author FROM books WHERE author = %s`, print rows, close.

Requirements

- Config dict (password from env if you can)
- `%s` + tuple — no f-string SQL
- `fetchall()` (or a `for` over the cursor)
- `try` / `except` on connector errors; `finally` close
- If `books` is empty, say so — do not crash

Expected Behavior

Author `"Luciano Ramalho"` (seed data) prints `Fluent Python`. A bad password prints a connection error, not an unhandled traceback.

Sample Run

```
('Fluent Python', 'Luciano Ramalho')
```

Constraints

No ORM. One connector. Parameterized queries only.

Bonus

Also list members with **no** loans (the seed's LEFT JOIN ticket in `starter/queries.sql`).

Mini-Project (Assignment): Student Database Manager

Why this matters: Files cannot safely share one `students` table across clerks. A menu CLI over MySQL is the same connect → cursor → parameterized CRUD every backend uses.

Lab kits: 02_mysql_library_seed (SQL practice / `training_db`) · 20_sql_injection_demo (parameter discipline)

Objective

Menu-driven CRUD on a `students` table. Every query uses `%s`. No passwords in source.

Story / Scenario

A campus office keeps roll, name, branch, and marks. Two clerks must not corrupt the same CSV. MySQL holds the shared table; Python is the desk CLI.

Requirements

Piece	Detail
Table	`students` (roll PK, name, branch, marks)
CLI	Menu: add, list, update, delete, exit — helper module is fine
Safety	Every query parameterized (`%s`)
UX	Clear message on duplicate roll; connector errors to console
Ship	`requirements.txt` + README with MySQL setup steps
Secrets	Password from env (document in README)

Expected Behavior

- Create / Read / Update / Delete each work from the menu
- No f-string SQL for user input anywhere
- Duplicate roll does not dump an unhandled traceback

Sample Run

```
1 Add  2 List  3 Update  4 Delete  5 Exit
Choice: 1
Roll: 101
Name: Ananya
Branch: CSE
Marks: 88.5
Inserted.
Choice: 2
(101, 'Ananya', 'CSE', Decimal('88.50'))
Choice: 5
Bye.
```

Bonus Challenges

- Search by branch with parameterized `=` or `LIKE`
- Wrap multi-step updates in try / `rollback` on failure
- Fix `20_sql_injection_demo` `broken/login_query.py` as a warm-up

Done when

- [ ] CRUD works from the menu
- [ ] No f-string SQL for user input
- [ ] README lists create-database + run steps
- [ ] No passwords in source

Interview Q&A

Q1. Why connect Python to MySQL instead of only JSON files?
Web and multi-user apps need centralized, concurrent, relational storage with SQL. Python handles app logic; MySQL stores durable structured data.

Q2. What is a cursor?
The object on a connection that executes SQL and retrieves rows (`fetchone` / `fetchall`).

Q3. Parameterized query vs string formatting?
Parameters send values separately from SQL, so typed input cannot become a command.

Q4. `commit` vs `rollback` ?
`commit` makes the transaction permanent. `rollback` undoes it — a transfer or checkout + stock change must succeed or fail together.

Q5. Where should DB passwords live?
Environment variables or a secret store — never committed source. Limited DB users in production, not `root` .

Q6. Connector vs “ORM” (name only)?
A connector executes SQL you write. An ORM maps classes to tables and generates SQL. This module stays on the connector. You do not need an ORM for the lab.

Cheat Sheet

```
connect(**config)           → session to MySQL
cursor()                    → execute + fetch
```

```
execute(sql, (values,))     → %s placeholders (MySQL)
fetchone() / fetchall()     → one row / all rows
commit()                    → save INSERT/UPDATE/DELETE
rollback()                  → undo on failure
close()                     → cursor and connection
```

Never `f"SELECT ... {user_input}"`. Pick one of `mysql-connector-python` or PyMySQL.

Completion Checklist

```
[ ] I can explain why apps outgrow files
[ ] I know connect → cursor → execute → commit → close
[ ] I can import the connector and describe config keys
[ ] I can write a %s query and a tuple of values
[ ] I can fetch rows and commit writes
[ ] I never build SQL with f-strings from user input
[ ] I completed the lab against the seed (or students table)
[ ] I attempted the student CLI mini-project
[ ] I can explain parameterized queries in an interview
```

What's Next?

You can now let Python talk to a shared MySQL database without handing attackers the SQL string.

Next, HTML (shared FRONTEND) is the page the user sees. Later, a web app will call the same connect → cursor → parameterized CRUD behind that page.